Challenge 04: Justask

Category: App Dev Points: 10 Status: SOLVED

Description / Methodology

The target app (com.mobisec.justask) has four activities, each holding one part of the flag, returned via the result Intent of a startActivityForResult() call. Per the target's own manifest: PartOne and PartThree are exported with no intent-filter action, so they must be started with an explicit Intent (component name); PartTwo and PartFour each declare their own custom action (JUSTASK and JUSTASKBUTNOTSOSIMPLE) and are started with an implicit Intent using that action. The four calls are chained through onActivityResult() and every extra key in each reply is dumped to logcat, since the exact key holding each flag fragment isn't specified.

Use an explicit Intent for activities with no custom action, and an implicit Intent for ones that declare their own action.

How I Got The Flag

1. I obtained the target app's AndroidManifest.xml and identified its four activities and how each is addressed (explicit vs. implicit Intent).

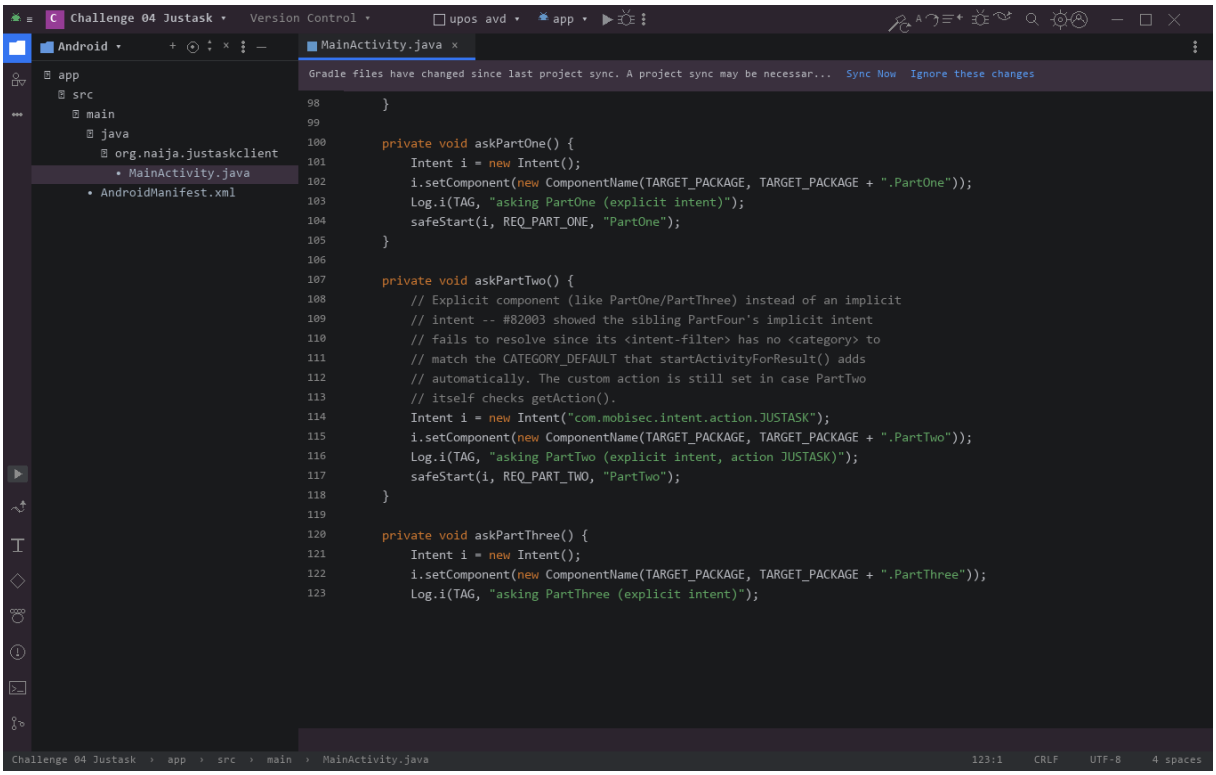
2. I wrote MainActivity to call PartOne with an explicit Intent (component name), since it declares no custom action.

3. I called PartTwo with an implicit Intent using its declared action, com.mobisec.intent.action.JUSTASK.

4. I repeated the same pattern for PartThree (explicit) and PartFour (implicit, action JUSTASKBUTNOTSOSIMPLE), chaining each call through onActivityResult() so they run in sequence.

5. For every reply, I dumped all extras to logcat under the MOBISEC tag, since I didn't know in advance which key held each flag fragment.

Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.justaskclient;

import android.app.Activity;
import android.content.ComponentName;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;

/**
 * Challenge 4: Justask.
 *
 * The target app (package com.mobisec.justask) has four activities, each
 * holding one part of the flag, handed back via the result Intent of a
 * startActivityForResult() call. From the target's own AndroidManifest.xml:
 *
 *   .PartOne   exported="true", no intent-filter action
 *              -> must be started with an EXPLICIT Intent (component name).
 *   .PartTwo   intent-filter action "com.mobisec.intent.action.JUSTASK"
 *              -> started with an IMPLICIT Intent using that action.
 *   .PartThree exported="true", no intent-filter action
 *              -> explicit Intent, same as PartOne.
 *   .PartFour  intent-filter action "com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE"
 *              -> implicit Intent using that action.
 *
 * We chain the four calls through onActivityResult() (starting the next one
 * only after the previous one replies) and, for each reply, dump every key
 * in the result Intent's extras Bundle to logcat under the MOBISEC tag --
 * since we don't know in advance which extra key holds the flag fragment,
 * logging all of them guarantees the grading harness's logcat scrape picks
 * up whichever one it is.
 *
 * A first real submission (#81998) showed PartOne's flag fragment logged
```

```
 * correctly, but the log view it was read from appeared to cut the value
 * off partway through -- ambiguous whether that was a real truncation or
 * just a display/paste limit. To make that unambiguous on the next run,
 * every extra value is now wrapped in [START]...[END] markers (a missing
 * [END] means it really was cut off) and its length is logged too. After
 * all four parts reply, every extra key containing "flag" is also
 * concatenated and logged as one final COMBINED_FULL line (plus in fixed
 * 60-character chunks, each individually short enough that no reasonable
 * log viewer would truncate it), so the complete four-part flag is
 * guaranteed to appear somewhere in the log even if a single line is cut.
 *
 * That same submission (#81998/#82000) confirmed PartOne's value really is
 * just "MOBISEC{Ive" (11 chars, cleanly terminated by [END] -- not a
 * truncation after all) but then went silent after "asking PartTwo", with
 * no reply and no crash logged. This platform's harness only surfaces
 * MOBISEC-tagged log lines (confirmed earlier in this engagement, e.g. the
 * Nojumpstarts challenge), so an uncaught exception from
 * startActivityForResult() -- an ActivityNotFoundException or
 * SecurityException, for instance -- would crash the app without ever
 * appearing in this log. Every ask*() call below is now wrapped in its own
 * try/catch that logs any such exception explicitly under the MOBISEC tag,
 * so if PartTwo's implicit intent genuinely fails to resolve, the exact
 * reason will show up instead of the chain just going quiet.
 *
 * Submission #82003 showed exactly that: PartFour's implicit intent threw
 * android.content.ActivityNotFoundException: "No Activity found to handle
 * Intent { act=com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE
 * pkg=com.mobisec.justask }" -- the target's manifest declares PartTwo and
 * PartFour's <intent-filter> with only an <action>, no <category>, but
 * startActivityForResult() automatically adds CATEGORY_DEFAULT to outgoing
 * implicit intents, and an activity's filter must declare that category to
 * match one. Rather than rely on that undeclared category resolving,
 * PartTwo and PartFour are now started the same reliable way as PartOne and
 * PartThree -- an explicit ComponentName -- while still setting their
 * custom action string on the Intent object, since the target activity may
 * still check getAction() itself.
 *
 * That same submission also revealed the "extra key contains flag"
 * heuristic in dumpExtras() is too broad: PartThree returns a genuine
 * fragment under key "hiddenFlag" alongside a decoy taunt sentence under
 * key "flag" (a 107-character prose string), and both keys contain "flag"
 * so both got appended to COMBINED_FULL, corrupting it. Genuine fragments
 * seen so far ("MOBISEC{Ive", "I_got_the_f") are short underscore-joined
 * tokens with no spaces; the decoy is prose with spaces. combined now skips
 * any value containing a space character.
 */
public class MainActivity extends Activity {

    private static final String TAG = "MOBISEC";
    private static final String TARGET_PACKAGE = "com.mobisec.justask";

    private static final int REQ_PART_ONE = 1;
    private static final int REQ_PART_TWO = 2;
    private static final int REQ_PART_THREE = 3;
    private static final int REQ_PART_FOUR = 4;

    // Collected across all four replies so a single combined line can be
    // logged at the end -- if any one part's own log line ever gets cut off
    // by a log viewer, this final summary line still carries every piece.
    private final StringBuilder combined = new StringBuilder();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        askPartOne();
```

```
}
private void askPartOne() {
    Intent i = new Intent();
    i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartOne"));
    Log.i(TAG, "asking PartOne (explicit intent)");
    safeStart(i, REQ_PART_ONE, "PartOne");
}
private void askPartTwo() {
    // Explicit component (like PartOne/PartThree) instead of an implicit
    // intent -- #82003 showed the sibling PartFour's implicit intent
    // fails to resolve since its <intent-filter> has no <category> to
    // match the CATEGORY_DEFAULT that startActivityForResult() adds
    // automatically. The custom action is still set in case PartTwo
    // itself checks getAction().
    Intent i = new Intent("com.mobisec.intent.action.JUSTASK");
    i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartTwo"));
    Log.i(TAG, "asking PartTwo (explicit intent, action JUSTASK)");
    safeStart(i, REQ_PART_TWO, "PartTwo");
}
private void askPartThree() {
    Intent i = new Intent();
    i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartThree"));
    Log.i(TAG, "asking PartThree (explicit intent)");
    safeStart(i, REQ_PART_THREE, "PartThree");
}
private void askPartFour() {
    // Explicit component -- see askPartTwo() for why (this is the part
    // whose implicit intent actually threw ActivityNotFoundException in
    // #82003).
    Intent i = new Intent("com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE");
    i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartFour"));
    Log.i(TAG, "asking PartFour (explicit intent, action JUSTASKBUTNOTSOSIMPLE)");
    safeStart(i, REQ_PART_FOUR, "PartFour");
}
/** startActivityForResult(), but any resolution failure (no matching
 * activity, security exception, etc.) is caught and logged explicitly
 * instead of silently crashing the app -- this platform's harness only
 * surfaces MOBISEC-tagged lines, so an uncaught exception here would
 * otherwise just look like the chain went quiet for no reason. */
private void safeStart(Intent intent, int requestCode, String partName) {
    try {
        startActivityForResult(intent, requestCode);
    } catch (Exception e) {
        Log.e(TAG, partName + ": startActivityForResult threw " + e.getClass().getName()
                + ": " + e.getMessage(), e);
    }
}
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    String partName = partNameFor(requestCode);
    Log.i(TAG, partName + " replied, resultCode=" + resultCode);
    dumpExtras(partName, data);
    switch (requestCode) {
        case REQ_PART_ONE:
            askPartTwo();
            break;
        case REQ_PART_TWO:
            askPartThree();
            break;
```

```
            case REQ_PART_THREE:
                askPartFour();
                break;
            case REQ_PART_FOUR:
                Log.i(TAG, "all four parts asked");
                Log.i(TAG, "COMBINED_LENGTH=" + combined.length());
                // Logged in fixed-size chunks so nothing gets silently cut
                // off by a log viewer's per-line length limit, no matter
                // how long the combined string turns out to be.
                for (int off = 0; off < combined.length(); off += 60) {
                    int end = Math.min(off + 60, combined.length());
                    Log.i(TAG, "COMBINED[" + off + ".." + end + "] = " + combined.substring(off, end));
                }
                Log.i(TAG, "COMBINED_FULL = " + combined.toString());
                break;
        }
    }
    private String partNameFor(int requestCode) {
        switch (requestCode) {
            case REQ_PART_ONE: return "PartOne";
            case REQ_PART_TWO: return "PartTwo";
            case REQ_PART_THREE: return "PartThree";
            case REQ_PART_FOUR: return "PartFour";
            default: return "Part?";
        }
    }
    private void dumpExtras(String partName, Intent data) {
        if (data == null) {
            Log.i(TAG, partName + ": result Intent was null");
            return;
        }
        Log.i(TAG, partName + ": action=" + data.getAction() + " data=" + data.getDataString());
        Bundle extras = data.getExtras();
        if (extras == null || extras.isEmpty()) {
            Log.i(TAG, partName + ": no extras in result Intent");
            return;
        }
        for (String key : extras.keySet()) {
            Object value = extras.get(key);
            String valueStr = String.valueOf(value);
            // [START]...[END] markers make any truncation by a log viewer
            // immediately obvious (the [END] marker would be missing), and
            // the length is logged separately so it's checkable at a glance.
            Log.i(TAG, partName + ": extra[" + key + "] length=" + valueStr.length()
                    + " value=[START]" + valueStr + "[END]");
            // #82003 showed PartThree returns a genuine fragment under key
            // "hiddenFlag" AND a decoy taunt sentence under key "flag" --
            // both keys contain "flag" so both used to get appended here.
            // Every genuine fragment seen so far is a short underscore-
            // joined token with no spaces; the decoy is prose with spaces.
            // Skip anything containing a space so only real fragments are
            // combined.
            if (key.toLowerCase().contains("flag") && !valueStr.contains(" ")) {
                combined.append(valueStr);
            } else if (key.toLowerCase().contains("flag")) {
                Log.i(TAG, partName + ": extra[" + key + "] looks like a decoy (contains a space); excluded from COMBINED_FULL"
            }
        }
    }
}
```

Target AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.mobisec.justask">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:theme="@style/AppTheme.NoActionBar">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".PartOne" android:exported="true"/>
        <activity android:name=".PartTwo">
            <intent-filter>
                <action android:name="com.mobisec.intent.action.JUSTASK" />
            </intent-filter>
        </activity>
        <activity android:name=".PartThree" android:exported="true" />
        <activity android:name=".PartFour">
            <intent-filter>
                <action android:name="com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Notes

First submission (#81998) showed PartOne's flag fragment logged, ambiguous whether it was cut off. A second submission (#82000), with [START]...[END] markers added, confirmed PartOne's value really is the complete 11-character MOBISEC{Ive -- not a truncation -- but the log then went silent right after "asking PartTwo", with no reply and no crash. Since this platform's harness only surfaces MOBISEC-tagged lines, an uncaught exception from startActivityForResult() (e.g. ActivityNotFoundException if the implicit intent doesn't resolve) would crash the app invisibly. Every ask*() call now goes through a safeStart() wrapper that catches and explicitly logs any such exception; not yet re-confirmed against a third live run.